Laboratory Report: Distributed Object Storage & Fault Tolerance Testing

Experiment Title: Distributed Object Storage & Fault Tolerance Testing
Data ： 2026-6-12
Class: Applied Computing
Name: Lu Weichi
ID: 202383910009

I. Experimental Purpose

Master Object Storage: Understand the difference between traditional file paths and S3-compatible Bucket/Object structures.
Experience Fault Tolerance: Observe how data remains accessible even when physical “disks” are destroyed.

II. Experiment Contents

In this experiment, a distributed cloud object storage system similar to Amazon S3 or Alibaba Cloud OSS was deployed using MinIO and Docker Compose. The following tasks were performed:

Deploy a 4- disk distributed MinIO cluster.
Upload and manage cloud objects.
Observe backend storage fragmentation.
Simulate a storage node failure.
Verify distributed storage fault tolerance.

III. Detailed Steps and Observations

Step 1: Create Experiment Directory and Enter It

In the Codespaces terminal, executed:

```
mkdir minio-lab && cd minio-lab
```

Step 2: Create Four Storage Directories (Simulate Disks)

Executed:

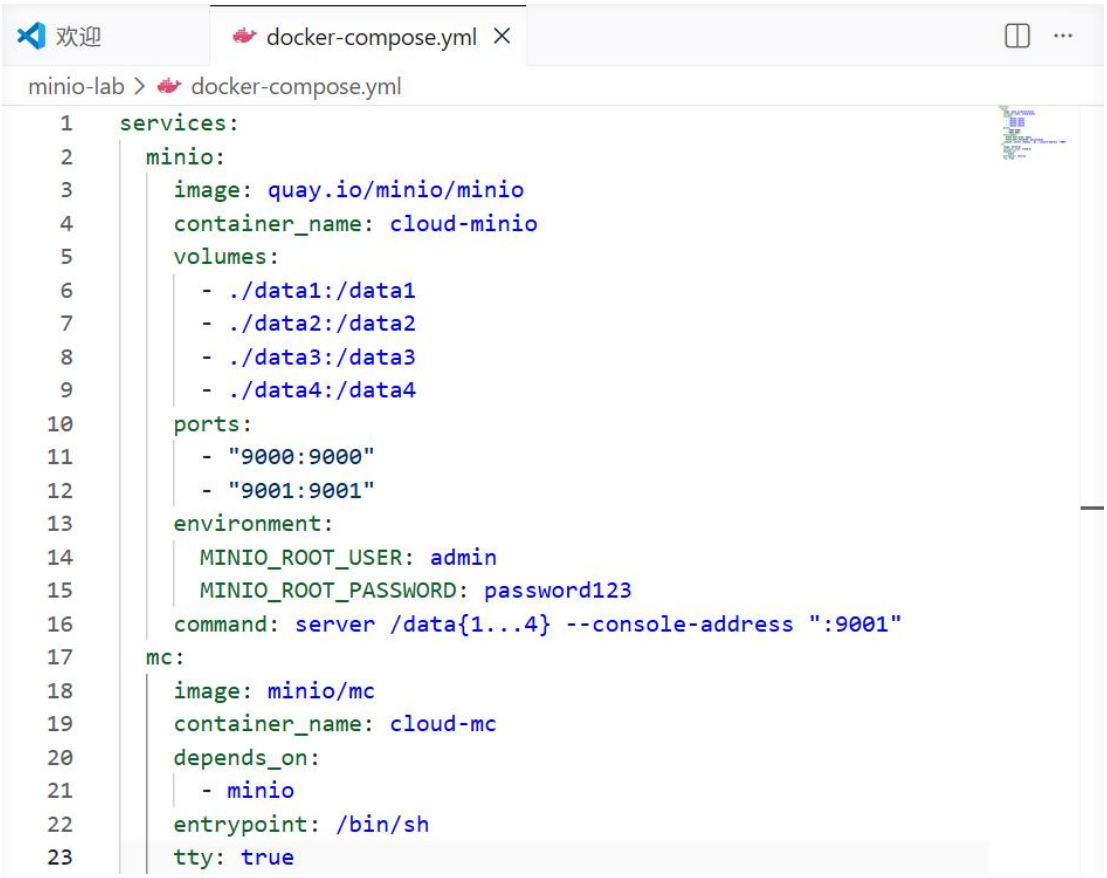
```
mkdir -p data1 data2 data3 data4ls
```

```
@lwc1116 →/workspaces/CloudComputing/minio-lab (main) $ mkdir -p data1 data2 dat
a3 data4
ls
data1  data2  data3  data4
@lwc1116 →/workspaces/CloudComputing/minio-lab (main) $
```

Step 3: Create docker-compose.yml File

Created the file docker-compose.yml in minio-lab with the following content:

services: minio: image: quay.io/minio/minio container_name: cloud-minio volumes: - ./data1:/data1 - ./data2:/data2 - ./data3:/data3 - ./data4:/data4 ports: - "9000:9000" - "9001:9001" environment: MINIO_ROOT_USER: admin MINIO_ROOT_PASSWORD: password123 command: server /data{1...4} --console-address ":9001" mc: image: minio/mc container_name: cloud-mc depends_on: - minio entrypoint: /bin/sh tty: true



Step 4: Explanation of docker-compose.yml Components (written in report)

Component	Meaning
image	Docker image used to create the container
container_name	Name of the container
volumes	Mount host directories into the container for persistent storage

Component	Meaning
`ports`	Port mapping: 9000 (API), 9001 (web console)
`environment`	Set MinIO root username and password
`command`	Start MinIO distributed server on four disks and enable console
`depends_on`	Service dependency – ensures MinIO starts before the client container

Step 5: Launch the Distributed Cluster

Executed:

`docker compose up -ddocker ps`

```
@lwc1116 ➜/workspaces/CloudComputing/minio-lab (main) $ docker compose up -d

[+] Running 3/3
 ✓ Network minio-lab_default  Created                                    0.1s
 ✓ Container cloud-minio      Started                                    3.9s
 ✓ Container cloud-mc         Started                                    1.2s
@lwc1116 ➜/workspaces/CloudComputing/minio-lab (main) $
```

```
@lwc1116 ➜/workspaces/CloudComputing/minio-lab (main) $ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          ST
ATUS          PORTS                                                             N
AMES
d6b6c5cf6085   minio/mc               "/bin/sh"                21 seconds ago   Up
 20 seconds                                                                      c
loud-mc
157557ee6c61   quay.io/minio/minio    "/usr/bin/docker-ent…"   24 seconds ago   Up
 21 seconds   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp   c
loud-minio
@lwc1116 ➜/workspaces/CloudComputing/minio-lab (main) $
```

Step 6: Configure MinIO Client Alias

Ran:

`docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123`

Output: `Added myminio successfully.`

```
@lwc1116 ➜/workspaces/CloudComputing/minio-lab (main) $ docker exec cloud-mc mc
alias set myminio http://minio:9000 admin password123
Added `myminio` successfully.
```

Step 7: Create a Bucket

Executed:

```
docker exec cloud-mc mc mb myminio/my-cloud-storage
```

```
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $ docker exec cloud-mc mc
mb myminio/my-cloud-storage
Bucket created successfully `myminio/my-cloud-storage`.
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $
```

Step 8: Create and Upload a Large Test File

8.1 Generate a 20 MB random binary file

```
dd if=/dev/urandom of=bigfile.bin bs=1M count=20
```

```
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $ dd if=/dev/urandom of=bi
gfile.bin bs=1M count=20
20+0 records in
20+0 records out
20971520 bytes (21 MB, 20 MiB) copied, 0.0898568 s, 233 MB/s
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $
```

8.2 Verify file size

```
ls -lh bigfile.bin
```

```
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $ ls -lh bigfile.bin
-rw-rw-rw- 1 codespace codespace 20M Jun 12 07:34 bigfile.bin
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $
```

8.3 Copy the file into the cloud-mc container

```
docker cp bigfile.bin cloud-mc:/bigfile.bin
```

```
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $ docker cp bigfile.bin cl
oud-mc:/bigfile.bin
Successfully copied 21MB to cloud-mc:/bigfile.bin
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $
```

8.4 Upload the file to MinIO

```
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
```

```
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $ docker exec cloud-mc mc
cp /bigfile.bin myminio/my-cloud-storage/
`/bigfile.bin` -> `myminio/my-cloud-storage/bigfile.bin`
```

Total	Transferred	Duration	Speed
20.00 MiB	20.00 MiB	00m00s	27.52 MiB/s

```
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $
```

Step 9: Verify Object Upload

Executed:

```
docker exec cloud-mc mc ls myminio/my-cloud-storage/
```

Output shows bigfile.bin with size 20 MiB.

```
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $ docker exec cloud-mc mc
ls myminio/my-cloud-storage/
[2026-06-12 08:23:59 UTC]  20MiB STANDARD bigfile.bin
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $
```

Step 10: Observe Backend Storage Fragmentation

Ran:

```
ls -R ./data1ls -R ./data2ls -R ./data3ls -R ./data4
```

Observation: Each data directory contains subdirectories (e.g., my-cloud-storage) and files (e.g., xl.meta). The file is split into multiple fragments distributed across the four disks, demonstrating MinIO's erasure coding.

```
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $ ls -R ./data1
ls -R ./data2
ls -R ./data3
ls -R ./data4
my-cloud-storage

./data4/my-cloud-storage:
bigfile.bin

./data4/my-cloud-storage/bigfile.bin:
5164d1dd-8d3e-4f93-8fe1-b054d9c4e805  xl.meta

./data4/my-cloud-storage/bigfile.bin/5164d1dd-8d3e-4f93-8fe1-b054d9c4e805:
part.1  part.2
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $
```

Step 11: Simulate Disk Failure

11.1 Stop the cluster

```
docker compose down
```

```
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $ docker compose down
[+] Running 3/3
 ✓ Container cloud-mc          Removed                                  10.2s
 ✓ Container cloud-minio       Removed                                   0.2s
 ✓ Network minio-lab_default   Removed                                   0.1s
@lwc1116 ➜ /workspaces/CloudComputing/minio-lab (main) $
```

11.2 Delete one storage directory (simulate disk loss)

sudo rm -rf data3mkdir data3

```
@lwc1116 →/workspaces/CloudComputing/minio-lab (main) $ sudo rm -rf data3
mkdir data3
@lwc1116 →/workspaces/CloudComputing/minio-lab (main) $
```

11.3 Restart the cluster

docker compose up -d

```
@lwc1116 →/workspaces/CloudComputing/minio-lab (main) $ docker compose up -d
[+] Running 3/3
 ✓ Network minio-lab_default  Created                                    0.1s
 ✓ Container cloud-minio      Started                                    0.4s
 ✓ Container cloud-mc         Started                                    0.6s
@lwc1116 →/workspaces/CloudComputing/minio-lab (main) $
```

Step 12: Verify Fault Tolerance

Checked whether the object is still accessible:

docker exec cloud-mc mc ls myminio/my-cloud-storage/

Result: bigfile.bin is still listed and accessible.

```
@lwc1116 →/workspaces/CloudComputing/minio-lab (main) $ docker exec cloud-mc mc
ls myminio/my-cloud-storage/
[2026-06-12 08:52:07 UTC]  20MiB STANDARD bigfile.bin
@lwc1116 →/workspaces/CloudComputing/minio-lab (main) $
```

IV. Reflection

1. Problems Encountered and Proposed Solutions

Problem	Cause	Solution
`docker exec` failed because container `cloud-mc` was not running	The container had been stopped earlier	Restarted the whole cluster with `docker compose up -d`
`mc ls` showed "Requested path not found" after restart	The MinIO client alias `myminio` was lost after container restart	Re-configured the alias using `docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123`

2. Explain why data is still available after deleting one storage directory.

Answer:

MinIO uses **erasure coding** to split a file into multiple data blocks and parity blocks, distributing them

across different disks. With four disks, the system can tolerate the loss of one disk (or even two, depending on configuration). When a disk is removed or its contents are cleared, MinIO automatically reconstructs the missing fragments using the remaining data and parity blocks from the other disks. This mechanism ensures that the object remains fully accessible without any manual intervention, demonstrating the high availability and fault tolerance of distributed object storage systems.